TASK 10

CONTAINERIZING A FLASK DEEP LEARNING API WITH DOCKER

Name: Monu Ajay Rai

Student ID: 5783524

Roll no: 27

Subject: Deep Learning from Production to Deployment

GitHub URL: https://github.com/monu8097/Task_10_Docker_Flask_API.git

Table of Contents

1. Objective
2. Introduction
3. Tools & Technologies
4. Project Structure
5. Flask API Implementation
6. Dockerfile Explanation
7. Docker Image Creation
8. Container Execution
9. API Health Check
10. Prediction Testing
11. Docker Logs
12. Image Optimization
13. Image History Analysis
14. Optimized Image
15. Testing Results
16. Observations
17. Screenshots
18. Conclusion

Appendix A: Source code listings

Appendix B: Docker command references

Appendix C: Troubleshooting notes

Appendix D: Suggestions for future work

Appendix E: Glossary

Abstract

This report documents Task 10, **Containerizing Flask Deep Learning API**, in which a trained Convolutional Neural Network (CNN) for CIFAR-10 image classification is exposed through a Flask REST API and then packaged into a Docker container. The purpose of the exercise is to show that a deep learning model can be delivered as a portable, self-contained and reproducible unit that behaves the same way on any machine that can run Docker.

The application consists of a single Python file, `flask_api.py`, which loads the model `cifar10_cnn.keras` once at start-up and exposes two routes: a health-check route (`GET /`) and a prediction route (`POST /predict`). The prediction route accepts an uploaded image, converts it to RGB, resizes it to 32 × 32 pixels, normalizes the pixel values to the range 0 to 1, and passes it to the model. The response is a small JSON document holding the predicted class name and the confidence expressed as a percentage.

A Dockerfile based on `python:3.11-slim` was written to install the Python dependencies, copy the application and the model, expose port 5000 and start the server. The image `task10-flask-api:latest` was built in 207.1 seconds and reported a disk usage of 3.11 GB with a content size of 710 MB. A container was started with the port mapping 5000:5000, its logs were inspected, its health-check route was verified, and a frog photograph was submitted with `curl.exe`. The API returned the class **Frog** with a confidence of **99.01 %**.

The image history was then analysed. The `pip install` layer of 2.25 GB was identified as the dominant contributor to the size of the image, whereas the model layer is only 3.84 MB. An image tagged `task10-flask-api:optimized` was rebuilt using the optimized Dockerfile practices (slim base image, `--no-cache-dir`, dependency-first copy order and a `.dockerignore` file). The rebuild finished in 2.1 seconds because every step was served from the layer cache, and a second container running on host port 5001 also passed the health check. The report presents the design, the commands, the evidence in the form of screenshots, the observations and the conclusions of the task.

1. Objective

The main objective of Task 10 is to package a Flask-based deep learning API into a Docker container so that it can be deployed reliably. The task statement lists the approach as creating a Dockerfile, building a Docker image, running the container locally, verifying the API functionality and optimizing the image size. The deliverables are the Dockerfile, the Docker image and a deployment report, and the evaluation criteria are correct containerization, successful deployment and the use of optimization practices.

1.1 Primary Objectives

- **Containerize the Flask deep learning API** so that the application, its Python dependencies and the trained model travel together as one unit.
- **Create a Docker image** from a Dockerfile that is readable, ordered for cache efficiency and reproducible.
- **Run the API inside a Docker container** and expose it to the host machine through a port mapping.
- **Test the API with a CIFAR-10 image** and verify that the returned class and confidence are meaningful.
- **Inspect the Docker container logs** to confirm the start-up sequence and to understand every warning that TensorFlow prints.
- **Apply Docker image optimization practices** and justify each of them.
- **Compare the original image with the optimized image** using `docker images` and `docker image history`.
- **Verify that the optimized image also runs successfully** in a separate container.

1.2 Deliverables

Table 1: Deliverables of Task 10

Deliverable	What was produced	Where it is described
Dockerfile	Six-instruction Dockerfile based on `python:3.11-slim`	Section 6 and Appendix A
Docker Image	`task10-flask-api:latest` and `task10-flask-api:optimized`	Sections 7, 12 and 14
Deployment report	This document with commands, outputs and screenshots	Whole report
Source code	`flask_api.py`, `requirements.txt`, `.dockerignore`	Section 5 and Appendix A
Evidence	13 screenshots plus the test image and architecture diagram	Sections 5 to 14 and Section 17

1.3 Evaluation Criteria Mapping

The evaluation criteria of the task are correct containerization, successful deployment and optimization practices. Table 2 shows how each criterion is addressed by this report so that a reviewer can locate the evidence quickly.

Table 2: Mapping of evaluation criteria to report sections

Criterion	Evidence in this report	Sections
Correct containerization	Dockerfile explanation, successful image build, image contents shown in the image history	6, 7, 13
Successful deployment	Running container, port mapping, container logs, health check and CIFAR-10 prediction	8, 9, 10, 11
Optimization practices	Slim base image, no pip cache, layer caching, `.dockerignore`, optimized image rebuild and comparison	12, 13, 14

1.4 Scope of the Task

The scope of the task is deliberately limited to deployment. The CIFAR-10 CNN model was trained in earlier tasks of the course, and the Flask API was developed and tested locally before this task began. Task 10 therefore does not attempt to retrain the model or to improve its accuracy. It concentrates on the questions that arise after a model exists: how to describe the runtime environment, how to package the model with its code, how to start it in a predictable way, and how to observe and measure the result.

Equally, the task does not attempt to build a production-grade serving stack. The Flask development server is used inside the container, which is adequate for demonstrating containerization and for local testing. Section 16 and Appendix D discuss the steps that would be required to move from this demonstration to a production deployment, such as a WSGI server, pinned dependency versions and health probes.

2. Introduction

Building a machine learning model is only one part of delivering a machine learning product. A model that works inside a notebook has to be wrapped in an interface, installed with the correct library versions, started with the correct command and made reachable from other programs. Each of these steps is a place where a working model can fail on another machine. The commonly heard sentence "it works on my machine" describes exactly this problem.

Containerization is the standard answer to this problem. A container packages an application together with everything it needs to run: the operating-system user space, the language runtime, the libraries, the configuration and the application files. Because the container carries its own environment, the behaviour of the application no longer depends on what happens to be installed on the host. Docker is the most widely used tool for creating and running such containers.

2.1 Background of the Project

In the previous tasks of the course a CNN was trained on the CIFAR-10 dataset and saved in the Keras format as `cifar10_cnn.keras`. A Flask application, `flask_api.py`, was then written to load the saved model and to expose it through HTTP. The API accepts an image file, applies the same preprocessing that was used during training and returns the predicted class. In this task the same application is moved into a Docker container.

2.2 The CIFAR-10 Classification Problem

CIFAR-10 is a benchmark dataset of small colour images. It contains 60,000 images of 32 × 32 pixels that belong to ten mutually exclusive classes: airplane, automobile, bird, cat, deer, dog, frog, horse, ship and truck. Because the images are small, the dataset is convenient for experiments and for teaching, and CNNs achieve good accuracy on it with modest computing resources.

The model that is deployed in this task follows that setting. It receives a tensor of shape (1, 32, 32, 3) and returns ten probabilities that sum to one. The position of the largest probability is the predicted class index, and the value at that position is used as the confidence of the prediction.

2.3 What is a REST API for a Model?

A REST API allows other programs to use the model without knowing anything about TensorFlow or Python. A client sends an HTTP request, and the server answers with a JSON document. In this project the client uploads an image using a multipart form field called `image`, and the server answers with the fields `predicted_class` and `confidence`. Any language that can send an HTTP request, for example a web page, a mobile application or a shell command, can therefore use the model.

2.4 What is Docker?

Docker builds and runs containers. A **Dockerfile** is a text file that lists the steps required to build an image. An **image** is a read-only, layered template produced by those steps. A **container** is a running instance of an image with its own file system, network interface and process space. Containers are much lighter than virtual machines because they share the kernel of the host, but they are isolated from each other and from the host in the ways that matter for deployment.

Table 3: Key Docker terms used in this report

Term	Meaning
Image	Read-only, layered template that contains the file system and metadata needed to start a container
Container	A running instance of an image with an isolated file system, network and process space
Layer	Result of one Dockerfile instruction; layers are cached and shared between images
Registry	A service such as Docker Hub that stores and distributes images
Build context	The set of files sent to the Docker builder when `docker build` is executed
Port mapping	A rule that forwards a host port to a container port, written as `host:container`
Tag	A human-readable label such as `latest` or `optimized` attached to an image

2.5 Why Containerize a Deep Learning API?

- **Reproducibility:** the Python version, TensorFlow build and system libraries are fixed inside the image, so the API behaves the same on every machine.
- **Portability:** the same image can run on a laptop, a test server or a cloud service that supports containers.
- **Isolation:** the heavy TensorFlow dependency does not pollute the Python environment of the host and cannot conflict with other projects.
- **Simple deployment:** starting the service is a single `docker run` command instead of a list of manual installation steps.
- **Scalability:** because a container is a unit that can be started repeatedly, several instances can be run side by side, which is the starting point for load balancing.

2.6 Report Organization

The remaining sections follow the order in which the task was carried out. Section 3 lists the tools and technologies, Section 4 shows the project structure and the architecture, and Section 5 explains the Flask API. Sections 6 to 11 describe the Dockerfile, the image, the container, the health check, the prediction test and the logs. Sections 12 to 14 deal with optimization, the image history analysis and the optimized image. Sections 15 to 18 present the test results, the observations, the screenshot index and the conclusion. The appendices contain the source listings, a command reference, troubleshooting notes, suggestions for future work and a glossary.

3. Tools & Technologies

The project combines a small number of well-known tools. This section describes each of them and explains the role it plays in the task.

Table 4: Technologies used in Task 10

Technology	Role in the project
Python	Programming language of the API; the container uses Python 3.11 from the `python:3.11-slim` image
Flask	Lightweight web framework that provides the routes `/` and `/predict`
TensorFlow / Keras	Loads the `.keras` model and performs inference
NumPy	Array handling for the image tensor and the prediction vector
Pillow	Opens the uploaded image, converts it to RGB and resizes it
Docker	Builds images and runs containers
Docker Desktop	Docker engine and graphical interface on the Windows development machine (`docker:desktop-linux` builder)
Docker CLI	Commands `build`, `run`, `ps`, `logs`, `images` and `image history`
CIFAR-10 dataset	Ten-class image dataset used to train the CNN
VS Code	Editor and integrated terminal used for development and for running the commands
curl.exe	Command-line HTTP client used to upload the test image

3.1 Python

Python is the language in which the API, the preprocessing code and the model were written. The Docker image uses Python 3.11, and the image history in Section 13 shows the exact version that was installed by the base image, 3.11.16. The Python installation in the slim image is complete enough to run pip, Flask and TensorFlow but omits many of the extra tools that are included in the full Python image.

3.2 Flask

Flask is a micro-framework for web applications. It provides routing, request parsing and JSON responses with very little code. For a model-serving API these features are sufficient: one route reports that the service is alive, and one route receives an uploaded file and returns a result. Flask also makes it easy to run the same code locally and inside a container, because the only difference is the address on which the server listens.

3.3 TensorFlow and Keras

TensorFlow is the machine learning framework that executes the model. Keras is its high-level API, and the `.keras` file is Keras' native save format. Loading it with `tf.keras.models.load_model` restores the architecture and the trained weights in a single call. TensorFlow is also the reason why the image is large: the pip package and its dependencies account for most of the size of the image, as the analysis in Section 13 shows.

3.4 NumPy and Pillow

Pillow reads the bytes of the uploaded image and can convert and resize them. NumPy converts the resulting image into an array, and the array operations used in preprocessing (type conversion, division and adding a batch dimension) are written with NumPy. NumPy is also used to find the index of the largest value in the prediction vector.

3.5 Docker and Docker Desktop

Docker is used to describe, build and run the container. On the Windows machine used for this task, Docker Desktop provides the Docker engine and the command-line client. The build output in Section 7 shows that the builder is `docker:desktop-linux`, which means that the image is built by a Linux-based engine even though the development machine runs Windows. This is one of the advantages of containers: the image that is produced is a Linux image and will behave in the same way on a Linux server.

3.6 VS Code and the Terminal

Visual Studio Code was used as the editor and, through its integrated terminal, as the place where the commands were run. The first screenshot of the report shows the editor with the project explorer on the left and the terminal below the code. The terminal is PowerShell, which explains the prompt `PS D:\Task10_Docker_Flask_API>` and the use of `curl.exe` (with the extension) instead of `curl`, because in PowerShell `curl` is an alias for another command.

3.7 Software Environment Summary

Table 5: Environment used for the task

Item	Value observed
Host operating system	Windows with PowerShell terminal
Project folder	`D:\Task10_Docker_Flask_API`
Builder	`docker:desktop-linux`
Base image	`python:3.11-slim` (Python 3.11.16 on Debian "trixie")
Working directory in container	`/app`
Exposed port	5000/tcp
Model file	`model/cifar10_cnn.keras`, saved by Keras 3.15.1
Python packages	Flask, tensorflow, numpy, Pillow (from `requirements.txt`)

4. Project Structure

The project is kept in a single folder named `Task10_Docker_Flask_API`. Keeping the application, the model, the Dockerfile and the documentation together makes the folder a self-contained unit that can be built with one command from its root. The layout is shown below and was also visible in the file explorer of VS Code in the first screenshot.

```
Task10_Docker_Flask_API/
|
|-- model/
|   `-- cifar10_cnn.keras
|
|-- screenshots/
|   |-- 01_flask_api_local_running.png
|   |-- 02_flask_health_check.png
|   |-- 03_Dockerfile.png
|   |-- 04_docker_build_success.png
|   |-- 05_docker_image_created.png
|   |-- 06_docker_container_running.png
|   |-- 07_docker_container_logs.png
|   |-- 08_docker_api_health_check.png
|   |-- 09_docker_prediction_result.png
|   |-- 10_docker_image_optimization_history.png
|   |-- 11_optimized_image_build.png
|   |-- 12_optimized_image_size_comparison.png
|   `-- 13_optimized_container_api_check.jpeg
|
|-- .dockerignore
|-- Dockerfile
|-- flask_api.py
|-- frog.png
|-- README.md
`-- requirements.txt
```

4.1 Role of Each File

Table 6: Files in the project and their purpose

File or folder	Purpose
`flask_api.py`	Flask application: loads the model, defines the health and prediction routes and starts the server
`model/cifar10_cnn.keras`	Trained CIFAR-10 CNN in the Keras format (about 3.8 MB)
`Dockerfile`	Recipe that describes how to build the image
`requirements.txt`	List of Python packages that must be installed: Flask, tensorflow, numpy and Pillow
`.dockerignore`	List of paths that must not be sent to the Docker build context
`frog.png`	Sample image used to test the prediction endpoint
`screenshots/`	Evidence images captured while running the task

File or folder	Purpose
`README.md`	Documentation of the project; excluded from the image by `.dockerignore`

4.2 The Model Directory

The model is stored in the folder `model/` and is copied into the image with the instruction `COPY model ./model`. The Python code refers to the path `model/cifar10_cnn.keras`, which is relative to the working directory. Because the Dockerfile sets the working directory to `/app` and copies the folder to `/app/model`, the same relative path works both on the development machine and inside the container. Using a relative path is a small design decision that removes the need for any code change when the application is containerized.

4.3 Architecture of the Deployed System

The diagram below shows the system as it is deployed. The Docker container runs the Flask application. The application dispatches GET requests to the health endpoint and POST requests to the prediction endpoint. The prediction endpoint prepares the image, submits a batch to the CIFAR-10 model, receives the scores, maps the index of the best score to a class name and returns a JSON response to the API client.

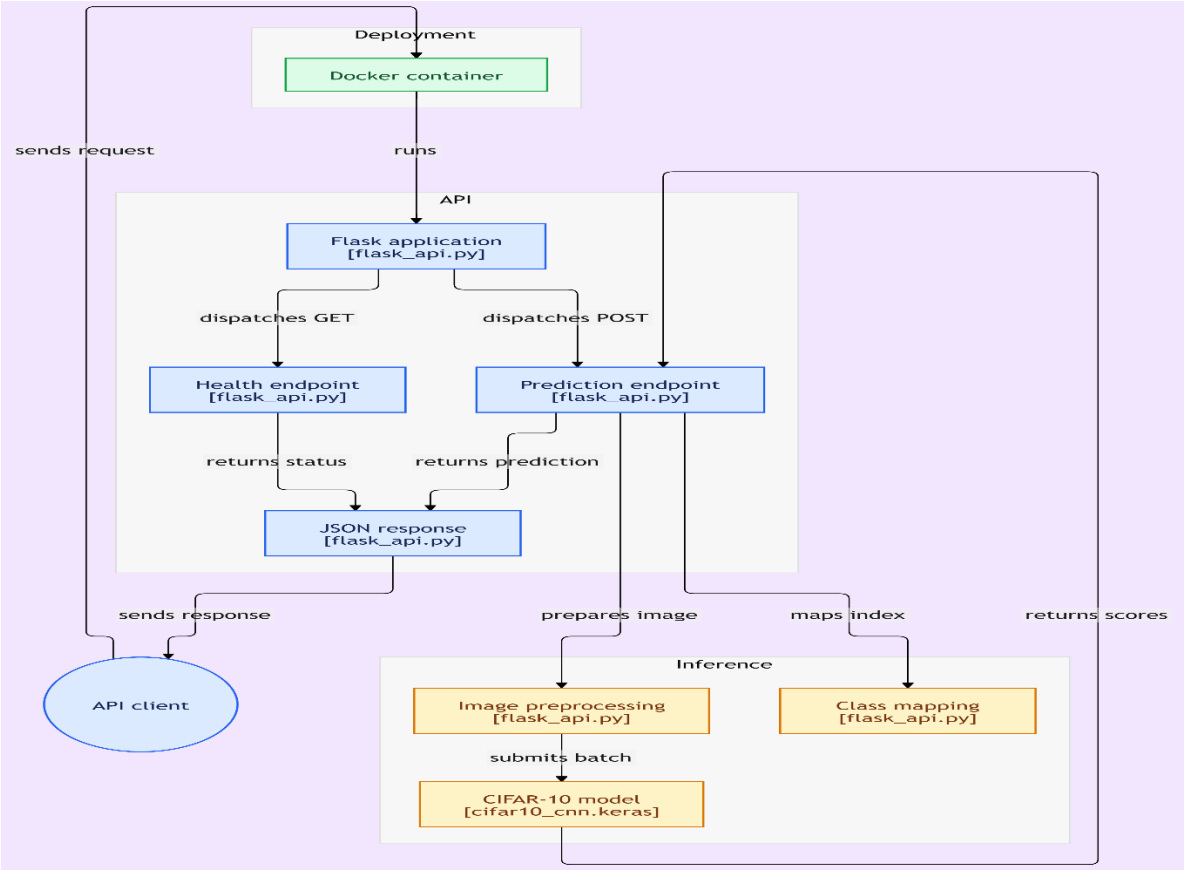


Figure 1: Architecture of the containerized CIFAR-10 Flask API

The diagram groups the components into three areas. The **Deployment** area contains the Docker container, which is the runtime boundary. The **API** area contains the Flask application, the two endpoints and the JSON response, all of which are implemented in `flask_api.py`. The **Inference** area contains the image preprocessing, the class mapping and the CIFAR-10 model file `cifar10_cnn.keras`. The arrows labelled "sends request" and "sends response" show the round trip of a single prediction from the client to the container and back.

4.4 Request Flow in Words

1. The API client sends an HTTP request to port 5000 of the host. Docker forwards it to port 5000 of the container.
2. The Flask application receives the request and dispatches it to a handler according to the method and the path.
3. A GET request to `/` is answered by the health endpoint with a short status document.
4. A POST request to `/predict` is answered by the prediction endpoint, which reads the uploaded image from the form field `image`.
5. The image is converted to RGB, resized to 32 × 32, scaled to the range 0 to 1 and given a batch dimension.
6. The batch is submitted to the CIFAR-10 model, which returns ten class scores.
7. The class with the highest score is selected and mapped to its name, and the score is converted to a percentage.
8. The result is returned to the client as a JSON response.

5. Flask API Implementation

The API is implemented in a single file, `flask_api.py`. The file is organized in clearly marked blocks: imports, application creation, class names, model loading, the health-check endpoint, the prediction endpoint and the entry point. This section explains each block and then shows the API running locally, before it was containerized.

5.1 Imports and Application Object

```
from flask import Flask, request, jsonify
import tensorflow as tf
import numpy as np
from PIL import Image
import io

app = Flask(__name__)
```

Flask supplies the application object, the `request` object that gives access to the uploaded file, and `jsonify`, which turns a Python dictionary into a JSON response with the correct content type. TensorFlow loads the model, NumPy prepares the array, and Pillow (imported as `PIL`) reads the image. The module `io` is imported but the final code opens the file object directly, so it is not strictly needed.

5.2 Class Names

```
class_names = [
    "Airplane", "Automobile", "Bird", "Cat", "Deer",
    "Dog", "Frog", "Horse", "Ship", "Truck"
]
```

The list holds the ten CIFAR-10 class names in the order of the class indices used during training. The order is essential: index 6 is "Frog", and the model was trained with the same ordering. If the order in this list differed from the order used in training, the API would return wrong names even when the numeric prediction was correct.

5.3 Loading the Trained Model

```
MODEL_PATH = "model/cifar10_cnn.keras"

model = tf.keras.models.load_model(MODEL_PATH)
```

The model is loaded once, when the module is imported, and not once per request. Loading a model involves reading the file and rebuilding the graph, which is slow compared with a single prediction. By keeping the model in a module-level variable, every request reuses the same loaded model, and the cost of loading is paid only at start-up. The consequence is that the container needs a few seconds after `docker run` before it can answer requests.

5.4 Architecture of the CNN Model

The configuration stored inside the `.keras` file shows that the model is a compact CNN with two convolution blocks followed by a small classifier. The file was saved by Keras 3.15.1. The architecture is summarized in Table 7, and the parameter counts are computed from the layer definitions.

Table 7: Layers of the CIFAR-10 CNN stored in `cifar10_cnn.keras`

Layer	Settings	Output shape	Parameters
Input	RGB image	(32, 32, 3)	0
Conv2D	32 filters, 3 × 3, ReLU	(30, 30, 32)	896
MaxPooling2D	2 × 2	(15, 15, 32)	0
Conv2D	64 filters, 3 × 3, ReLU	(13, 13, 64)	18,496
MaxPooling2D	2 × 2	(6, 6, 64)	0
Flatten	-	(2304)	0
Dense	128 units, ReLU	(128)	295,040
Dropout	rate 0.5	(128)	0
Dense	10 units, softmax	(10)	1,290
Total			315,722

The first convolution layer learns 32 small filters that respond to simple patterns such as edges and colour transitions. After pooling, the second convolution layer combines them into 64 more complex feature maps. The feature maps are flattened into a vector of 2,304 values, which the dense layer of 128 units transforms into a compact representation. Dropout with a rate of 0.5 is active only during training and randomly switches off half of these units to reduce over-fitting. The final layer has ten units with a softmax activation, which produces ten probabilities that sum to one.

With 315,722 parameters stored as 32-bit floats, the weights alone occupy roughly 1.26 MB. The model file is about 3.8 MB, which is approximately three times larger. This is consistent with the file also storing the optimizer state (two additional values per parameter for the Adam optimizer), and it explains why the model layer in the Docker image history is 3.84 MB. Either way, the model is tiny compared with the TensorFlow library that is needed to run it.

5.5 Health-Check Endpoint

```
@app.route("/", methods=["GET"])
def home():
    return jsonify({
        "message": "CIFAR-10 Flask Prediction API is running",
        "endpoint": "/predict",
        "method": "POST"
    })
```

The root route answers GET requests with a JSON document that states that the service is running and tells the caller how to use it. It performs no computation, so it is a quick way to check that the server is up and reachable. In this task it is used both locally and after containerization to confirm that the port mapping works.

5.6 Prediction Endpoint

```
@app.route("/predict", methods=["POST"])
def predict():
    if "image" not in request.files:
        return jsonify({
            "error": "No image provided. Please upload an image."
        }), 400

    try:
        image_file = request.files["image"]
        image = Image.open(image_file).convert("RGB")
        image = image.resize((32, 32))
        image_array = np.array(image)
        image_array = image_array.astype("float32") / 255.0
        image_array = np.expand_dims(image_array, axis=0)

        predictions = model.predict(image_array, verbose=0)
        predicted_index = np.argmax(predictions[0])
        predicted_class = class_names[predicted_index]
        confidence = float(predictions[0][predicted_index]) * 100

        return jsonify({
            "predicted_class": predicted_class,
            "confidence": round(confidence, 2)
        })
    except Exception as e:
        return jsonify({"error": str(e)}), 500
```

The route accepts only POST requests. It first checks whether the form field `image` is present in the uploaded files; if it is missing, it returns a JSON error with the HTTP status code 400 (Bad Request), which tells the client that the request itself was wrong. The remaining work is placed in a `try` block so that a corrupted or unsupported file produces a JSON error with status 500 instead of an unhandled exception.

Preprocessing steps

Table 8: Preprocessing applied to every uploaded image

Step	Operation	Reason
1	`Image.open(...)` reads the uploaded file	Decodes PNG, JPEG and other formats into a Pillow image
2	`.convert("RGB")`	Guarantees three channels even for grayscale or RGBA images
3	`.resize((32, 32))`	Matches the input size of the CIFAR-10 model
4	`np.array(image)`	Creates an array of shape (32, 32, 3)
5	`astype("float32") / 255.0`	Scales pixel values to the range 0 to 1, as during training
6	`np.expand_dims(..., axis=0)`	Adds a batch dimension so the shape is (1, 32, 32, 3)

Preprocessing must exactly match the preprocessing that was used to train the model. If the training data had been scaled to the range 0 to 1 and the API sent raw values from 0 to 255, the predictions would be meaningless. The identical scaling in training and in the API is therefore a correctness requirement, not just a convenience.

Interpreting the output

The call `model.predict(...)` returns an array of shape (1, 10). The expression `np.argmax(predictions[0])` finds the index of the largest probability, and `class_names[predicted_index]` turns it into a readable name. The confidence is that largest probability multiplied by 100 and rounded to two decimals. A confidence of 99.01 therefore means that the model assigned a probability of about 0.9901 to the winning class.

5.7 Application Entry Point

```
if __name__ == "__main__":
    app.run(
        host="0.0.0.0",
        port=5000,
        debug=False
    )
```

The host value `0.0.0.0` is the most important line for containerization. If Flask listened only on `127.0.0.1`, the server would accept connections only from inside the container itself, and the port mapping from the host would not work. Binding to `0.0.0.0` makes Flask listen on all network interfaces of the container, so traffic forwarded by Docker can reach it. Debug mode is switched off because the debugger allows code execution and must never be exposed on a network.

5.8 Running the API Locally

Before the application was placed in a container, it was run directly on the development machine to prove that the code and the model work. The screenshot below shows the end of `flask_api.py` in VS Code and the terminal output of the server. TensorFlow prints its start-up messages, followed by the Flask messages that show the server running on all addresses of the machine on port 5000.

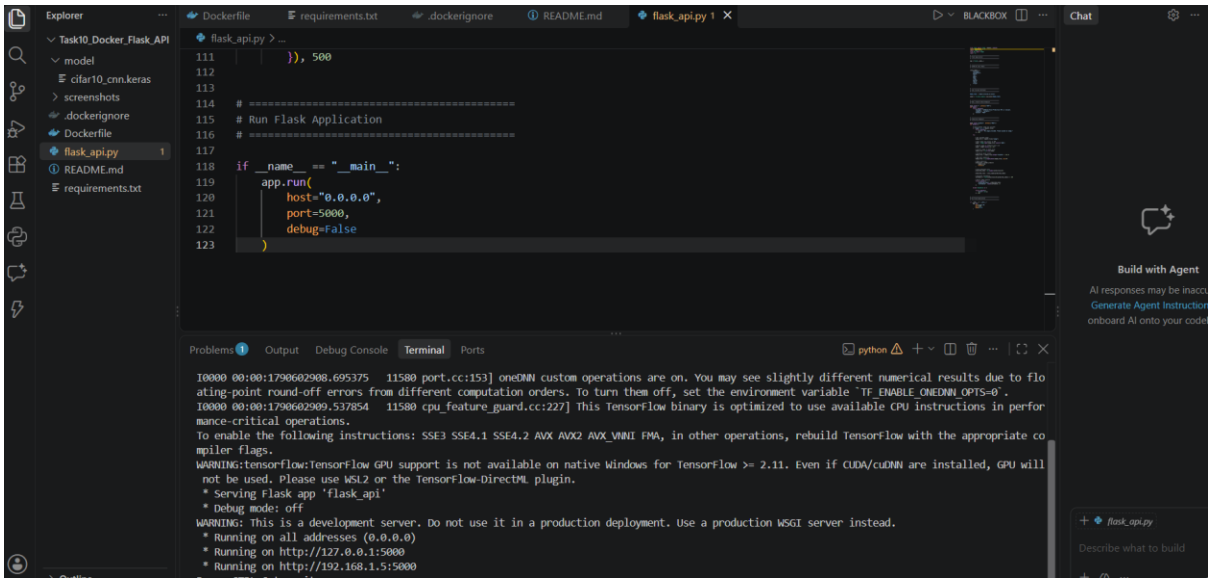


Figure 2: Flask API running locally in VS Code

The terminal output contains four messages that are worth understanding. The oneDNN message says that custom operations are enabled, which may cause tiny numerical differences because of floating-point round-off. The CPU feature message says that the TensorFlow binary uses the instruction sets SSE3, SSE4.1, SSE4.2, AVX, AVX2, AVX_VNNI and FMA on this CPU. The warning about GPU support explains that TensorFlow 2.11 and later does not support GPU execution on native Windows, so the model runs on the CPU. Finally, Flask reports that debug mode is off and that the server is running on 127.0.0.1:5000 and on the local network address 192.168.1.5:5000, together with the reminder that the built-in server is a development server.

5.9 Local Health Check

Opening `http://localhost:5000/` in a browser sends a GET request to the root route. The response is the JSON document that was defined in the `home` function.

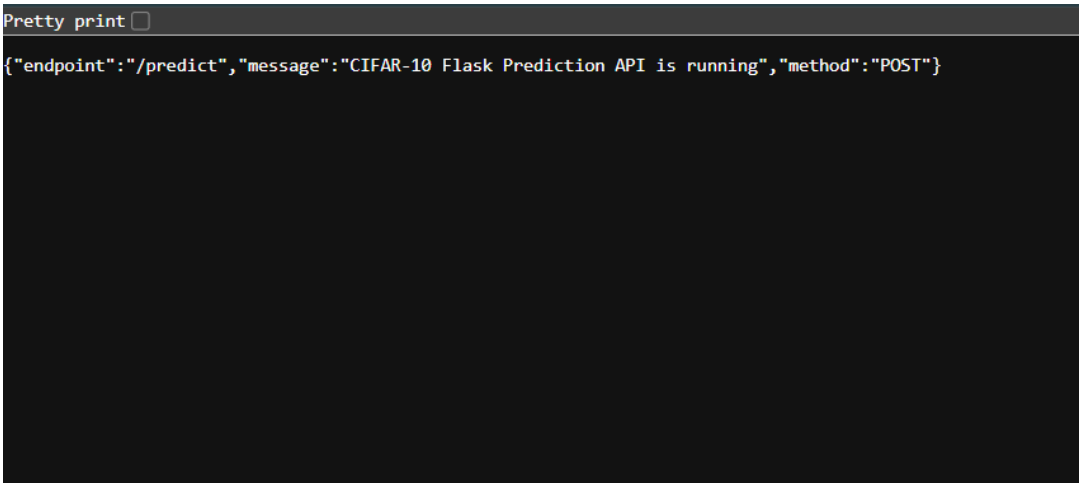


Figure 3: Local API health check in the browser

The response contains the three fields `endpoint`, `message` and `method`. Flask sorts the keys alphabetically in its JSON output, which is why `endpoint` appears first. The local test shows that the application itself is correct, so any later failure inside the container can be attributed to the container configuration rather than to the code.

6. Dockerfile Explanation

The Dockerfile is the central deliverable of the task. It is short, but each line was chosen deliberately. The full file is shown below and appears in the screenshot that follows it.

```
# Use a lightweight Python base image
FROM python:3.11-slim

# Set the working directory inside the container
WORKDIR /app

# Copy requirements first for better Docker layer caching
COPY requirements.txt .

# Install Python dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Copy Flask application
COPY flask_api.py .

# Copy trained deep learning model
COPY model ./model

# Expose Flask API port
EXPOSE 5000

# Start Flask API
CMD ["python", "flask_api.py"]
```

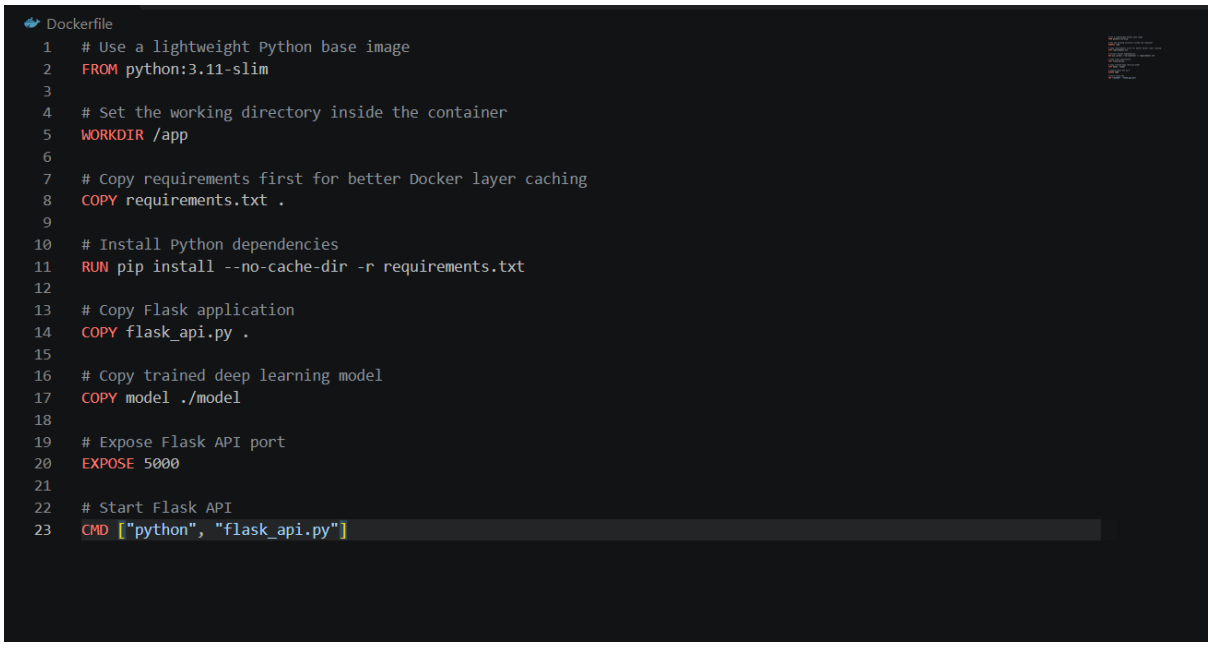


Figure 4: Dockerfile of the Flask deep learning API

6.1 Instruction-by-Instruction Explanation

Table 9: Dockerfile instructions and their purpose

Instruction	Explanation
`FROM python:3.11-slim`	Selects the base image. The slim variant contains Python 3.11 on a minimal Debian system, without compilers and documentation that are not needed at run time
`WORKDIR /app`	Creates the directory `/app` and makes it the current directory for the instructions that follow and for the running container
`COPY requirements.txt .`	Copies only the dependency list into `/app`. Doing this before copying the code is what makes layer caching effective
`RUN pip install --no-cache-dir -r requirements.txt`	Installs Flask, TensorFlow, NumPy and Pillow. The flag prevents pip from storing downloaded packages in its cache inside the layer
`COPY flask_api.py .`	Copies the application code. This layer changes most often, so it is placed after the dependency layer
`COPY model ./model`	Copies the trained model into `/app/model`, where the code expects to find it
`EXPOSE 5000`	Documents that the application listens on port 5000. It does not publish the port by itself
`CMD ["python", "flask_api.py"]`	Defines the default command that runs when a container starts. The exec form is used so that Python receives signals directly

6.2 Choice of the Base Image

The full `python` image is based on a larger Debian system with many development tools, and it is several hundred megabytes larger than the slim variant. The alpine variant is smaller still, but it uses a different C library (musl), and pre-built TensorFlow packages are built for the standard glibc, so installation on alpine is difficult or impossible. The slim variant is therefore a good compromise for a TensorFlow application: it is small, it uses glibc and it works with the pre-built pip wheels.

The image history in Section 13 shows what the base image contributes. The Debian layer is 87.6 MB, the Python installation layer is 48.8 MB, and the apt layer is 4.94 MB. Together they are about 141 MB, which is small compared with the 2.25 GB layer created by installing the Python packages.

6.3 The Importance of Instruction Order

Docker builds an image layer by layer, and it can reuse a layer from a previous build if the instruction and everything before it are unchanged. The order of the instructions in the Dockerfile has therefore a direct effect on build time. The dependency list changes rarely, whereas the application code changes often. By copying `requirements.txt` and installing the packages first, and copying the code afterwards, a change in

`flask_api.py` invalidates only the layers that come after the code copy. The expensive installation layer, which took 151.8 seconds in the first build, is then reused from the cache.

If the Dockerfile had instead used a single `COPY . .` before `pip install`, every edit of the code would have invalidated the installation layer, and TensorFlow would have been downloaded again at each build. This ordering is one of the most valuable practices in Docker and is covered again in Section 12.

6.4 EXPOSE, Port Publishing and the Host Address

`EXPOSE 5000` is documentation. It records the port on which the process is expected to listen, and tools can read it, but by itself it does not make the port reachable from the host. The port is published when the container is started with `-p 5000:5000`. Both settings must agree with the port used in `app.run`, and the application must bind to `0.0.0.0`, otherwise the forwarded connection would be refused.

6.4.1 The CMD Instruction

`CMD` sets the default process of the container. Written in the exec form, as a JSON array, it starts `python flask_api.py` directly, without an intermediate shell. This means that when Docker stops the container, the signal is delivered to the Python process itself, so the application can shut down cleanly. The container stops when this main process ends, which is why the Flask server must run in the foreground.

6.5 The requirements.txt File

```
Flask
tensorflow
numpy
Pillow
```

The file lists the four packages that the application imports. No version numbers are given, so pip installs the latest compatible releases at build time. This keeps the file simple and worked for this task. For a production system it is better practice to pin the versions (for example `Flask==3.0.3`) so that a rebuild in the future produces the same environment. This improvement is included in the suggestions in Appendix D.

6.6 The .dockerignore File

```
__pycache__
*.pyc
*.pyo
*.pyd
.git
.gitignore
.vscode
screenshots
screenshots9
README.md
```

When `docker build` runs, the client first sends the build context, which is the project folder, to the builder. Any file in the context can be copied into the image and, more importantly, any change to a file that is copied invalidates the layers that depend on it. The `.dockerignore` file excludes Python cache files, version-control data, editor settings, the screenshots and the README. The build output shows the effect: the context transferred for the first build was only 3.83 MB, which is essentially the model file, and the transfer of the `.dockerignore` context itself was 137 bytes.

7. Docker Image Creation

With the Dockerfile in place, the image was built from the project folder with the following command. The dot at the end of the command tells Docker to use the current directory as the build context.

```
docker build -t task10-flask-api:latest .
```

The option `-t` gives the image the name `task10-flask-api` and the tag `latest`. The build ran to completion, and the output is shown below.

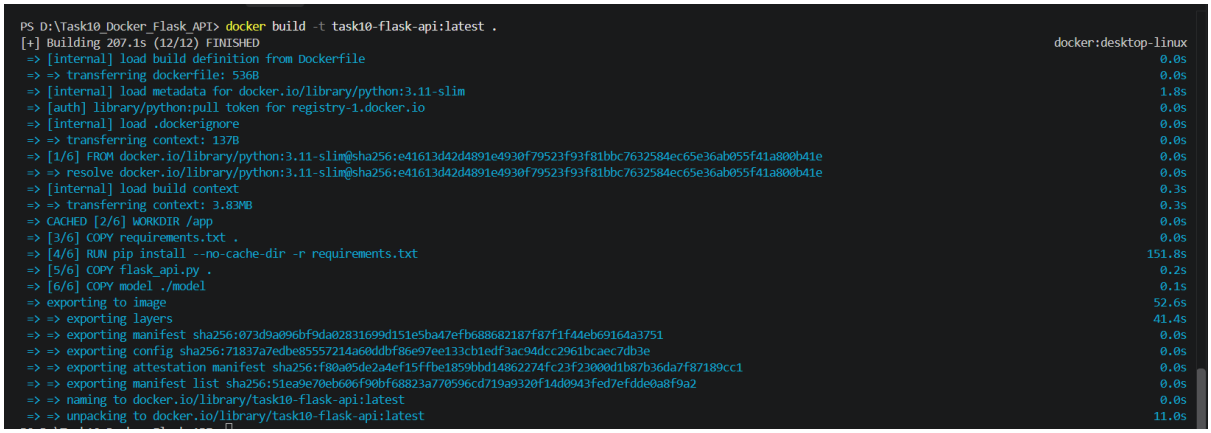


Figure 5: Docker build output for task10-flask-api:latest

7.1 Reading the Build Output

BuildKit prints one line for each internal step and the time that it took. The header shows that the build finished after 207.1 seconds, in 12 steps, using the builder `docker:desktop-linux`. The steps can be grouped as follows.

Table 10: Build steps and timings for task10-flask-api:latest

Step	Time	Comment
Load build definition from Dockerfile	0.0 s	Dockerfile of 536 bytes
Load metadata for `python:3.11-slim`	1.8 s	Checks the registry for the base image
Load `.dockerignore`	0.0 s	Build context description of 137 bytes
Load build context	0.3 s	Context of 3.83 MB (mainly the model)
[2/6] `WORKDIR /app`	0.0 s	CACHED: the layer already existed
[3/6] `COPY requirements.txt .`	0.0 s	Small file, copied instantly
[4/6] `RUN pip install --no-cache-dir -r requirements.txt`	151.8 s	Downloads and installs TensorFlow and other packages

Step	Time	Comment
[5/6] `COPY flask_api.py .`	0.2 s	Application code
[6/6] `COPY model ./model`	0.1 s	Trained model
Exporting to image	52.6 s	Including 41.4 s for exporting layers
Unpacking to `docker.io/library/task10-flask-api:latest`	11.0 s	Makes the image available in the local image store

The step that dominates the build is the installation of the Python packages: 151.8 seconds out of 207.1 seconds, which is about 73 % of the total time. The export of the image (52.6 seconds) is also long, and it is directly related to the size of the layers that must be written. The steps that involve the application code and the model take only a fraction of a second, which confirms that the design decision to place them after the installation step is sensible.

The build output also lists the manifest and the configuration digests that were created. The image ID that appears later in `docker images` is derived from the manifest list. Two builds that use the same layers and the same configuration share these digests, and this observation is used again in Section 14.

7.2 Verifying the Created Image

After the build, the command `docker images` lists the images that are stored locally. The screenshot below shows the new image together with two other images that were already present on the machine.

```
docker images
```



Figure 6: Docker image created: task10-flask-api:latest

Table 11: Images listed by docker images after the first build

Image	ID	Disk usage	Content size	Status
busybox:latest	fd7dc98638c8	6.81 MB	2.23 MB	Not in use
task10-flask-api:latest	51ea9e70eb60	3.11 GB	710 MB	New image
task9-flask-api:latest	d1b67645c540	3.56 GB	813 MB	In use

The new image, `task10-flask-api:latest`, has the ID `51ea9e70eb60`. Docker reports two size figures. The **disk usage** of 3.11 GB is the space that the unpacked image occupies in the local image store, while the **content size** of 710 MB is the compressed size of the image content, which is what would be

transferred to a registry. The image from the previous task, `task9-flask-api:latest`, is larger (3.56 GB and 813 MB), so the image of this task is already smaller than its predecessor, although the reasons for the difference were not investigated in this task.

7.3 Why is the Image So Large?

An image of 3.11 GB may look surprising for an application that consists of a 12 kB Python file and a 3.8 MB model. The reason is the dependency layer. TensorFlow is a large library: it contains compiled kernels for many operations, the Keras package, and dependencies such as protocol buffers, gRPC, h5py, ml-dtypes and TensorBoard. When these are installed, the resulting layer is 2.25 GB, as shown in the image history. Docker's content-size figure of 710 MB is smaller because the layers are stored in compressed form.

The important point is that this size is a property of the dependency, not a defect of the Dockerfile. Section 12 looks at what can and cannot be done about it.

8. Container Execution

An image is only a template. To use the API, a container has to be created from it and started. This was done with a single command that runs the container in the background, gives it a name and maps a host port to the port of the application.

```
docker run -d --name task10-flask-container -p 5000:5000 task10-flask-api:latest
```

8.1 Explanation of the Command

Table 12: Parts of the docker run command

Part	Meaning
`docker run`	Creates a new container from an image and starts it
`-d`	Detached mode: the container runs in the background and the terminal is free for other commands
`--name task10-flask-container`	Gives the container a readable name so that it can be referred to in `docker logs`, `docker stop` and other commands
`-p 5000:5000`	Publishes the port: traffic to port 5000 of the host is forwarded to port 5000 of the container
`task10-flask-api:latest`	The image from which the container is created

The port mapping has the form `host:container`. The left number is the port on the host machine, which is the port that a browser or `curl` uses. The right number is the port inside the container on which Flask listens. In this task both are 5000, so the container can be reached at `http://localhost:5000`. Because they are independent values, the host port could be changed without touching the application, and Section 14 uses this to run a second container on host port 5001.

8.2 Verifying that the Container is Running

The command `docker ps` lists the running containers. It was executed immediately after the container was started.

```
docker ps
```

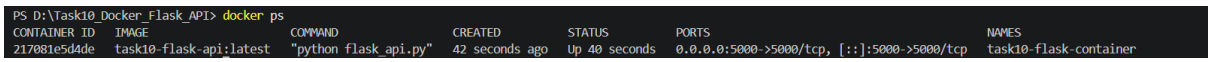


Figure 7: Running container listed by docker ps

Table 13: Values shown by docker ps for task10-flask-container

Column	Value observed
CONTAINER ID	`217081e5d4de`

Column	Value observed
IMAGE	`task10-flask-api:latest`
COMMAND	`"python flask_api.py"` (the CMD of the Dockerfile)
CREATED	42 seconds ago
STATUS	Up 40 seconds
PORTS	`0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp`
NAMES	`task10-flask-container`

The status "Up 40 seconds" confirms that the main process did not exit after starting, which is the first sign of a healthy container. The command column shows that the container is executing the command that was defined in the `CMD` instruction. The ports column shows the mapping in both IPv4 (`0.0.0.0`) and IPv6 (`[::]`) form: any address on the host at port 5000 is forwarded to port 5000 of the container.

8.3 Container Life Cycle

A container has a life cycle that is separate from the image. The image is never changed by running or deleting containers. The commands below summarize the operations that were used or that are useful when working with the container.

Table 14: Common container life-cycle commands

Command	Effect
`docker ps`	Lists running containers
`docker ps -a`	Lists all containers, including stopped ones
`docker logs task10-flask-container`	Prints the output of the main process
`docker stop task10-flask-container`	Sends a stop signal and waits for the process to end
`docker start task10-flask-container`	Starts a stopped container again
`docker rm task10-flask-container`	Deletes a stopped container
`docker rmi task10-flask-api:latest`	Deletes an image that is not used by any container

Because the name `task10-flask-container` is reserved as long as the container exists, a second container cannot be created with the same name. To recreate it, the old container must be removed first. This

is the reason why the optimized image in Section 14 is started under a different name and on a different host port.

9. API Health Check

A running container does not necessarily mean a working application. The process could be alive and still unable to answer requests, for example because the port mapping is wrong or because the model failed to load. A health check therefore goes one step further than `docker ps`: it sends a real request through the port mapping and checks the answer.

9.1 The Test

The root route was requested from the host machine at `http://localhost:5000/`. Because the container publishes port 5000, this request travels from the browser on the host, through Docker's port forwarding, to the Flask server inside the container.

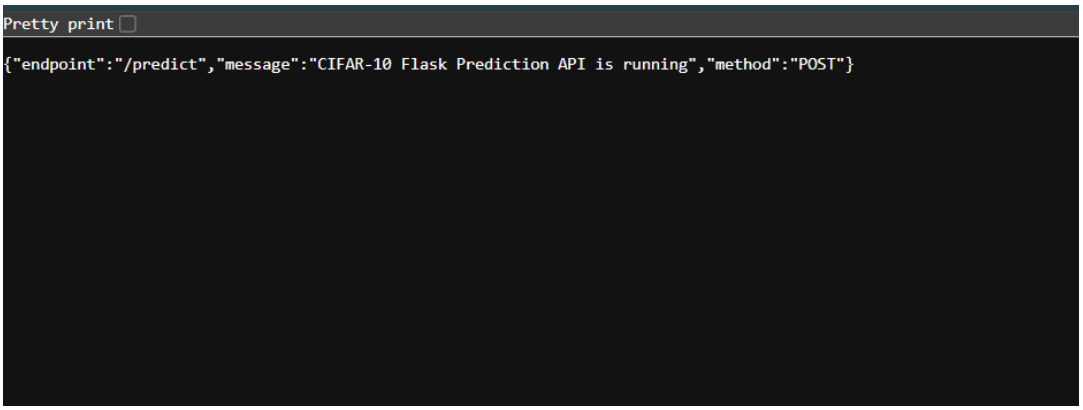


Figure 8: Health check of the containerized API at localhost:5000

9.2 Result

```
{
    "endpoint": "/predict",
    "message": "CIFAR-10 Flask Prediction API is running",
    "method": "POST"
}
```

The response is identical to the response that was obtained from the locally running application in Section 5. This is the essential proof of containerization: the same code now runs in a completely different environment and produces the same result. The response also confirms that the model was loaded successfully, because the Flask application would not have started if the call to `load_model` had failed.

9.3 What the Health Check Proves

- The container is running and its main process has not crashed.
- The Flask server is listening on `0.0.0.0` and not only on the loopback address of the container.
- The port mapping from host port 5000 to container port 5000 is correct.
- The Python dependencies were installed correctly, because the imports at the top of `flask_api.py` succeeded.
- The model file was copied to the expected path, because `load_model` succeeded during start-up.

The health check does not prove that the prediction route works. That is the purpose of the test in Section 10.

10. Prediction Testing

The most important test of the API is an actual prediction. A photograph of a frog, `frog.png`, was used as the test image. The image has a size of 611 × 407 pixels, which is much larger than the input size of the model, so the test also exercises the resizing step of the preprocessing.

10.1 The Test Image



Figure 9: Test image frog.png (611 x 407 pixels)

The picture shows a green frog with brown markings, photographed from the side against a white background. It is a clear, well-lit example, which makes it a reasonable test for a functional check. It differs from the CIFAR-10 training images in size and in style, because CIFAR-10 images are tiny 32 × 32 pixel images, but after resizing, the main colours and the shape of the animal are preserved.

10.2 The Request

The image was uploaded with `curl.exe`, using a multipart form request. The field name `image` in the command must match the key that the code reads from `request.files`.

```
curl.exe -X POST -F "image=@frog.png" http://localhost:5000/predict
```

Table 15: Parts of the curl command

Part	Meaning
`curl.exe`	The curl executable; the extension avoids the PowerShell alias called `curl`
`-X POST`	Uses the HTTP method POST, which the `/predict` route requires
`-F "image=@frog.png"`	Sends a multipart form field named `image` whose content is read from the file `frog.png`; the character `@` tells curl to upload the file content

Part	Meaning
`http://localhost:5000/predict`	Address of the prediction route through the published port

10.3 The Response

```
PS D:\Task10_Docker_Flask_API> curl.exe -X POST -F "image=@frog.png" http://localhost:5000/predict
{"confidence":99.01,"predicted_class":"Frog"}
```

Figure 10: Prediction result returned by the Docker container

```
{"confidence":99.01,"predicted_class":"Frog"}
```

The API returned the class **Frog** with a confidence of **99.01 %**. The predicted class is correct, and the model is very sure about it: nearly all of the probability mass is assigned to a single class. The two fields are exactly those that the `predict` function builds, and they appear in alphabetical order, as in the health-check response.

10.4 What Happened Inside the Container

1. curl sent the file to port 5000 of the host, and Docker forwarded the request into the container.
2. Flask matched the request to the `predict` function and found the file under the key `image`.
3. Pillow decoded the PNG file and converted it to RGB.
4. The 611 × 407 image was resized to 32 × 32 pixels.
5. The pixel values were converted to `float32` and divided by 255.
6. A batch dimension was added, giving the shape (1, 32, 32, 3).
7. The CNN computed ten probabilities, and the index of the largest one was 6.
8. `class_names[6]` is "Frog", and the value 0.9901 was converted to 99.01 %.
9. Flask returned the JSON response with the status code 200.

10.5 Interpreting the Confidence Value

The confidence value must be interpreted carefully. A softmax output is a measure of how strongly the model prefers one class over the others, and it does not guarantee that the prediction is correct. A model can be very confident and wrong, especially for images that are unlike its training data. In this test, the confidence of 99.01 % coincides with a correct answer, which supports the conclusion that the preprocessing in the container is consistent with the training pipeline. A single successful test, however, is a functional check and not an accuracy measurement. The accuracy of the model was evaluated on the CIFAR-10 test set in the earlier tasks.

10.6 Behaviour for Invalid Requests

The code contains two explicit error paths. If a POST request arrives without a form field named `image`, the API returns the message "No image provided. Please upload an image." with the status code 400. If an exception occurs while the image is being processed, for example because the uploaded file is not an image, the API returns the exception text with the status code 500. These paths were not part of the screenshots for this task, but they are visible in the source code in Section 5 and are the expected behaviour for invalid input.

11. Docker Logs

Everything that the main process of a container writes to its standard output or standard error is collected by Docker and can be read with `docker logs`. Logs are the first place to look when a container does not behave as expected, and reading them for a healthy container teaches what a normal start-up looks like.

```
docker logs task10-flask-container
```

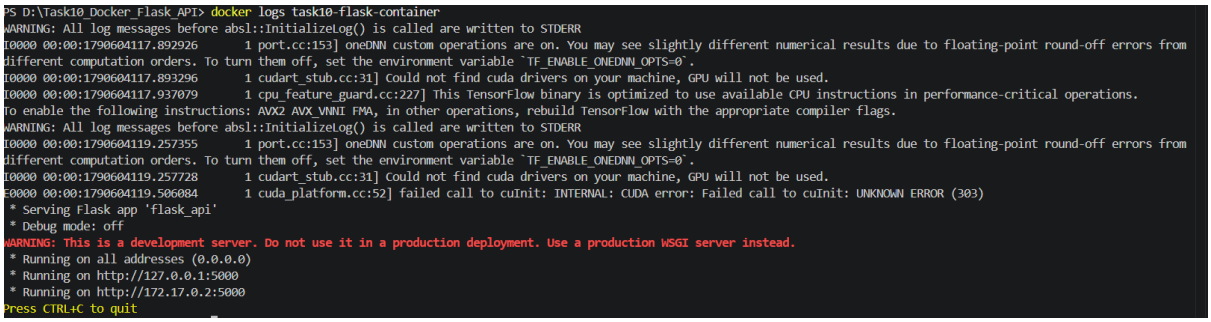


Figure 11: Docker logs of task10-flask-container

11.1 Line-by-Line Interpretation

Table 16: Messages found in the container logs

Message	Meaning
`All log messages before absl::InitializeLog() is called are written to STDERR`	Information from the Abseil logging library used by TensorFlow; it means that early messages go to the error stream
`oneDNN custom operations are on` (port.cc)	TensorFlow uses the oneDNN library for faster CPU maths; the results may differ very slightly because of floating-point round-off. The behaviour can be turned off with `TF_ENABLE_ONEDNN_OPTS=0`
`Could not find cuda drivers on your machine, GPU will not be used` (cudart_stub.cc)	No NVIDIA driver is available in the container, so TensorFlow runs on the CPU
`This TensorFlow binary is optimized to use available CPU instructions` (cpu_feature_guard.cc)	TensorFlow uses the instruction sets AVX2, AVX_VNNI and FMA of the processor for performance-critical operations
`failed call to cuInit: ... UNKNOWN ERROR (303)` (cuda_platform.cc)	TensorFlow tried to initialise CUDA and failed because there is no GPU; this is an error-level line, but it is harmless in a CPU-only deployment

Message	Meaning
`Serving Flask app 'flask_api'` and `Debug mode: off`	Flask has started the application from the module `flask_api` without the debugger
`WARNING: This is a development server...`	Flask reminds the user that the built-in server is not meant for production and that a WSGI server should be used instead
`Running on all addresses (0.0.0.0)`	The server is bound to all network interfaces, which is required for the port mapping to work
`Running on http://127.0.0.1:5000`	Loopback address inside the container
`Running on http://172.17.0.2:5000`	Address of the container on Docker's default bridge network
`Press CTRL+C to quit`	Standard Flask message; in detached mode the container is stopped with `docker stop`

11.2 Understanding the CUDA Messages

Three of the messages mention CUDA and could be mistaken for a problem. The TensorFlow package that was installed by pip contains code that tries to use an NVIDIA GPU if one is available. On this machine no GPU is available to the container, so the initialization fails and TensorFlow reports it. After reporting the failure, TensorFlow silently falls back to the CPU, and the application continues normally. The successful health check and the correct prediction in the earlier sections confirm that the messages do not affect the functionality of the API.

This is an important lesson for reading logs. A line that has the severity level E (error) is not always a fatal condition. The context, and in particular the messages that follow, must be used to decide whether action is needed. For this project the messages simply record that inference runs on the CPU. For a model as small as this CNN, CPU inference is fast enough for interactive use.

11.3 Comparison with the Local Run

The messages in the container logs are very similar to those seen in the terminal in Section 5. The container reports its address on the Docker bridge network (172.17.0.2), while the local run reported the address in the home network (192.168.1.5). The local run mentioned that GPU support is not available on native Windows, whereas the container reported that no CUDA drivers were found. The Flask messages are identical, which shows that the same application code is being executed in both places.

11.4 Useful Variations of the Logs Command

Table 17: Options of docker logs

Command	Effect
`docker logs -f task10-flask-container`	Follows the log output as it is produced
`docker logs --tail 20 task10-flask-container`	Shows only the last 20 lines
`docker logs --since 10m task10-flask-container`	Shows the messages of the last ten minutes
`docker logs -t task10-flask-container`	Adds a timestamp to each line

The Flask development server also writes one line to the log for every HTTP request that it handles. After the health check and the prediction test, those request lines can be found at the end of the log output when the command is repeated. They are a simple form of request auditing.

12. Image Optimization

The third evaluation criterion of the task is the use of optimization practices. In container work, optimization has two goals: to make the image small, so that it is quick to store, transfer and start, and to make the build fast, so that changes can be tested without long waiting times. The Dockerfile of this project applies four standard practices that address these goals.

12.1 Practices Applied

Table 18: Optimization practices used in the Dockerfile

Practice	Implementation	Benefit
Lightweight base image	`FROM python:3.11-slim`	Smaller starting point; fewer packages to store and to patch
No pip cache	`pip install --no-cache-dir`	Downloaded package archives are not kept in the layer
Layer caching	`COPY requirements.txt .` before `COPY flask_api.py .`	The 2.25 GB dependency layer is reused when only code changes
Build context control	`.dockerignore`	Small context (3.83 MB); unrelated files never invalidate the cache

12.2 Lightweight Base Image

The base image determines the smallest possible size of the final image, because every later layer is added on top of it. The slim variant of the official Python image keeps the Python runtime and the essential system libraries but omits build tools such as compilers and header files. Because TensorFlow is installed from a pre-built wheel, no compiler is needed during the build, and the smaller base image can be used without any loss of functionality.

12.3 Avoiding the pip Cache

By default pip stores every package that it downloads in a cache directory so that a later installation on the same machine can be faster. Inside a Docker build this cache has no value, because the layer is created once and the cache would only remain as dead weight inside the image. The flag `--no-cache-dir` disables the cache. For a large package such as TensorFlow, the archives that would otherwise be kept are hundreds of megabytes in size.

12.4 Layer Caching

Docker keeps the result of every instruction and reuses it when the instruction and its inputs are unchanged. The inputs of `COPY` are the files that it copies, so a change in one of those files invalidates the layer and all layers after it. Because `requirements.txt` is copied separately and before the code, editing the API code does not touch the dependency layer. The benefit is very large in this project, since the dependency layer takes 151.8 of the 207.1 seconds of a full build. Section 14 shows the effect: when nothing had changed, the whole rebuild took 2.1 seconds.

12.5 The .dockerignore File

A `.dockerignore` file has two effects. First, it reduces the amount of data that has to be sent to the builder at the start of every build. Second, it protects the cache: files that are not part of the application, such as screenshots and the README, can change without invalidating any layer. It also prevents accidental inclusion of files that should not be in an image, such as the `.git` folder that holds the entire version history.

12.6 Further Options and Their Trade-offs

Other techniques exist that could reduce the size or improve the quality of the image. They were not applied in this task because the goal was to keep the working configuration and to avoid reducing the size at the expense of functionality. They are listed here to show the range of possibilities and their costs.

Table 19: Additional optimization options not applied in this task

Option	Idea	Trade-off
CPU-only TensorFlow package	Install `tensorflow-cpu` instead of `tensorflow` because the container has no GPU	Needs a change in `requirements.txt` and a test; the saving must be measured
Pinned dependency versions	Write exact versions in `requirements.txt`	Improves reproducibility; does not change the size
Multi-stage build	Build in one stage and copy only the results into a clean final stage	Useful when compilation is needed; little gain when wheels are pre-built
Production WSGI server	Run the app with Gunicorn instead of the Flask development server	Adds a package; improves robustness and concurrency
Non-root user	Create and switch to an unprivileged user in the image	Better security; slightly more Dockerfile lines
TensorFlow Lite model	Convert the model and use a lightweight runtime	Large size reduction but requires model conversion and code changes

Each of these options is a design decision with consequences, so they are treated as suggestions for future work in Appendix D and not as part of the delivered solution.

13. Image History Analysis

Optimization starts with measurement. The command `docker image history` shows every layer of an image, the instruction that created it and its size. It is the tool that turns the statement "the image is large" into the more useful statement "this specific layer is large".

```
docker image history task10-flask-api:latest
```

```
PS D:\Task10_Docker_Flask_API> docker image history task10-flask-api:latest
IMAGE          CREATED         CREATED BY                                      SIZE      COMMENT
51ea9e70eb60   12 minutes ago   CMD ["python" "flask_api.py"]                   0B        buildkit.dockerfile.v0
<missing>      12 minutes ago   EXPOSE [5000/tcp]                               0B        buildkit.dockerfile.v0
<missing>      12 minutes ago   COPY model ./model # buildkit                   3.84MB    buildkit.dockerfile.v0
<missing>      12 minutes ago   COPY flask_api.py . # buildkit                  12.3kB    buildkit.dockerfile.v0
<missing>      12 minutes ago   RUN /bin/sh -c pip install --no-cache-dir -r…   2.25GB    buildkit.dockerfile.v0
<missing>      15 minutes ago   COPY requirements.txt . # buildkit              12.3kB    buildkit.dockerfile.v0
<missing>      2 days ago       WORKDIR /app                                    8.19kB    buildkit.dockerfile.v0
<missing>      9 days ago       CMD ["python3"]                                 0B        buildkit.dockerfile.v0
<missing>      9 days ago       RUN /bin/sh -c set -eux;  for src in idle3 p…   16.4kB    buildkit.dockerfile.v0
<missing>      9 days ago       RUN /bin/sh -c set -eux;  savedAptMark="$(a…    48.8MB    buildkit.dockerfile.v0
<missing>      9 days ago       ENV PYTHON_SHA256=91bcdebfdde239a003ae93738a…   0B        buildkit.dockerfile.v0
<missing>      9 days ago       ENV PYTHON_VERSION=3.11.16                      0B        buildkit.dockerfile.v0
<missing>      9 days ago       ENV GPG_KEY=A035C8C19219BA821ECEA86B64E628F8…   0B        buildkit.dockerfile.v0
<missing>      9 days ago       RUN /bin/sh -c set -eux;  apt-get update;  a…   4.94MB    buildkit.dockerfile.v0
<missing>      9 days ago       ENV LANG=C.UTF-8                                0B        buildkit.dockerfile.v0
<missing>      9 days ago       ENV PATH=/usr/local/bin:/usr/local/sbin:/usr…   0B        buildkit.dockerfile.v0
<missing>      10 days ago      # debian.sh --arch 'amd64' out/ 'trixie' '@1…   87.6MB    debuerreotype 0.17
```

Figure 12: Docker image history of task10-flask-api:latest

13.1 Layers Created by the Project

Table 20: Layers created by the project Dockerfile

Created by	Instruction	Size	Share
Project	`CMD ["python", "flask_api.py"]`	0 B	0 %
Project	`EXPOSE 5000/tcp`	0 B	0 %
Project	`COPY model ./model`	3.84 MB	0.16 %
Project	`COPY flask_api.py .`	12.3 kB	< 0.01 %
Project	`RUN pip install --no-cache-dir -r requirements.txt`	2.25 GB	about 94 %
Project	`COPY requirements.txt .`	12.3 kB	< 0.01 %
Project	`WORKDIR /app`	8.19 kB	< 0.01 %

The percentages are calculated relative to the sum of all layer sizes shown in the history, which is about 2.4 GB. The table makes the situation clear. The layer produced by `pip install` is 2.25 GB, which is roughly 94 % of the total. The layer that holds the model is 3.84 MB, and the layer that holds the application is 12.3 kB. Together the model and the code make up well under one percent of the image.

13.2 Layers Inherited from the Base Image

Table 21: Layers inherited from python:3.11-slim

Instruction (shortened)	Size	Purpose
Debian root file system created by `debian.sh` (trixie, amd64)	87.6 MB	Operating system files
`RUN` (apt-get update and install of base packages)	4.94 MB	System packages needed by Python
`RUN` (build and installation of Python, `savedAptMark`)	48.8 MB	Python 3.11 runtime
`RUN` (removal of `idle3` and related files)	16.4 kB	Cleanup step
`ENV PYTHON_VERSION=3.11.16`	0 B	Records the Python version
`ENV PYTHON_SHA256`, `ENV GPG_KEY`, `ENV LANG`, `ENV PATH`	0 B	Environment settings of the base image
`CMD ["python3"]`	0 B	Default command of the base image

The base image contributes about 141 MB in total (87.6 MB + 4.94 MB + 48.8 MB, plus a few kilobytes). This confirms that the choice of `python:3.11-slim` was efficient: even if the base image could be made smaller still, the possible saving would be small compared with the size of the dependency layer. The version line also shows that the runtime inside the container is Python 3.11.16, and the comment column shows that the layers of the project were created by BuildKit, the modern Docker builder.

13.3 Reading the Age Column

The CREATED column reveals which layers are new and which are inherited. The layers of the project were created "12 minutes ago", when the image was built. The layers of the base image were created between 9 and 10 days earlier, when the maintainers of the Python image published it. This shows a key property of layered images: the base layers are downloaded once and shared, and only the layers that the project adds are built locally.

13.4 Zero-Byte Layers

Several layers, such as `EXPOSE`, `CMD` and `ENV`, have a size of 0 B. These instructions change only the metadata of the image (its configuration) and not its file system, so they add no content. The Dockerfile can therefore contain as many of them as is useful for clarity without any effect on the size.

13.5 Conclusions of the Analysis

- The dependency installation layer of 2.25 GB is by far the largest contributor to the size of the image.
- The model layer of 3.84 MB and the code layer of 12.3 kB are negligible in size.

- The base image contributes about 141 MB, which is already small because of the slim variant.
- The largest reduction in size can therefore come only from changing the dependencies, for example by selecting a smaller TensorFlow distribution, and not from changing the application files.
- The sum of the layer sizes in the history (about 2.4 GB) differs from the 3.11 GB reported as disk usage by `docker images`; the two commands measure size differently and the difference was not investigated further in this task.

14. Optimized Image

After the analysis, the image was rebuilt with the optimized configuration and given the tag `optimized`. The purpose was to compare the result with the original image and to verify that the optimized image also works when it is run as a container.

14.1 Building the Optimized Image

```
docker build -t task10-flask-api:optimized .
```

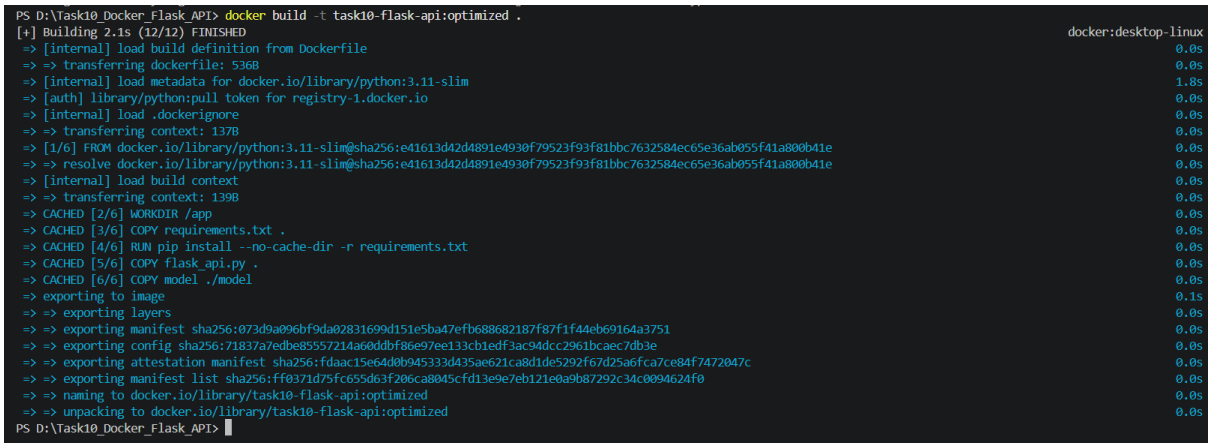


Figure 13: Build output of task10-flask-api:optimized

The build finished in 2.1 seconds, compared with 207.1 seconds for the first build. The output shows why. Every application step, from `WORKDIR /app` to `COPY model ./model`, is marked **CACHED**, which means that Docker found identical layers from the previous build and reused them. The only work that remained was loading the build definition, resolving the metadata of the base image (1.8 seconds) and exporting the image manifests.

Table 22: Comparison of the two builds

Item	task10-flask-api:latest	task10-flask-api:optimized
Total build time	207.1 s	2.1 s
`RUN pip install` step	151.8 s	CACHED (0.0 s)
`COPY flask_api.py .` step	0.2 s	CACHED (0.0 s)
`COPY model ./model` step	0.1 s	CACHED (0.0 s)
Exporting layers	41.4 s	0.0 s
Transferred build context	3.83 MB	139 bytes
Manifest digest	`sha256:073d9a09...`	`sha256:073d9a09...`
Configuration digest	`sha256:71837a7e...`	`sha256:71837a7e...`

The last two rows of the table are especially informative. The manifest digest and the configuration digest of the two builds are the same. The manifest digest identifies the list of layers and the configuration digest identifies the image configuration, so an identical value means that the two images consist of exactly the same layers with the same settings. Only the attestation manifest and the manifest list, which contain build metadata, have different digests. The optimized image is therefore the same image content under a second tag, which is exactly what one expects when the Dockerfile had already been optimized.

14.2 Image Size Comparison

```
docker images
```



Figure 14: Comparison of latest and optimized image sizes

Table 23: Images after building the optimized image

Image	ID	Disk usage	Content size	In use
busybox:latest	fd7dc98638c8	6.81 MB	2.23 MB	No
task10-flask-api:latest	51ea9e70eb60	3.11 GB	710 MB	Yes
task10-flask-api:optimized	ff0371d75fc6	3.11 GB	710 MB	Yes
task9-flask-api:latest	d1b67645c540	3.56 GB	813 MB	Yes

The two images of Task 10 have different IDs, `51ea9e70eb60` and `ff0371d75fc6`, but the same disk usage of 3.11 GB and the same content size of 710 MB. The different IDs are explained by the different manifest lists mentioned above. Because the layers are shared, Docker stores them only once, so having two tags for the image does not double the space that is needed for the layers.

14.3 Why the Size Did Not Decrease

The optimized image is not smaller than the original image, and this is the correct outcome. The original Dockerfile had already been written with the standard optimization practices: a slim base image, no pip cache, the dependency-first copy order, a `.dockerignore` file and a minimal build context. Rebuilding the same Dockerfile under a new tag cannot make the image smaller, because there is nothing left to remove without changing the application.

It would have been possible to show a smaller number by removing something, for example by dropping the model or replacing TensorFlow with a different library. That would have produced a smaller image but a broken or different application, and would not have demonstrated optimization in a meaningful way. For this reason, the optimization in this task is evaluated through the Dockerfile practices and the image-layer analysis, and not through an artificial reduction of the reported size.

14.4 Running the Optimized Container

To confirm that the optimized image is functional, a second container was started. The first container was still running on host port 5000, so the second container was given a different name and a different host port.

```
docker run -d --name task10-flask-optimized -p 5001:5000 task10-flask-api:optimized
```

Table 24: Differences between the two run commands

Setting	Original container	Optimized container
Image	`task10-flask-api:latest`	`task10-flask-api:optimized`
Name	`task10-flask-container`	`task10-flask-optimized`
Host port	5000	5001
Container port	5000	5000
URL	`http://localhost:5000/`	`http://localhost:5001/`

Only the host port changes. The container port remains 5000, because that is the port on which the unchanged application listens. This shows that the port mapping decouples the address seen from outside from the address used inside.

14.5 Verification of the Optimized Container

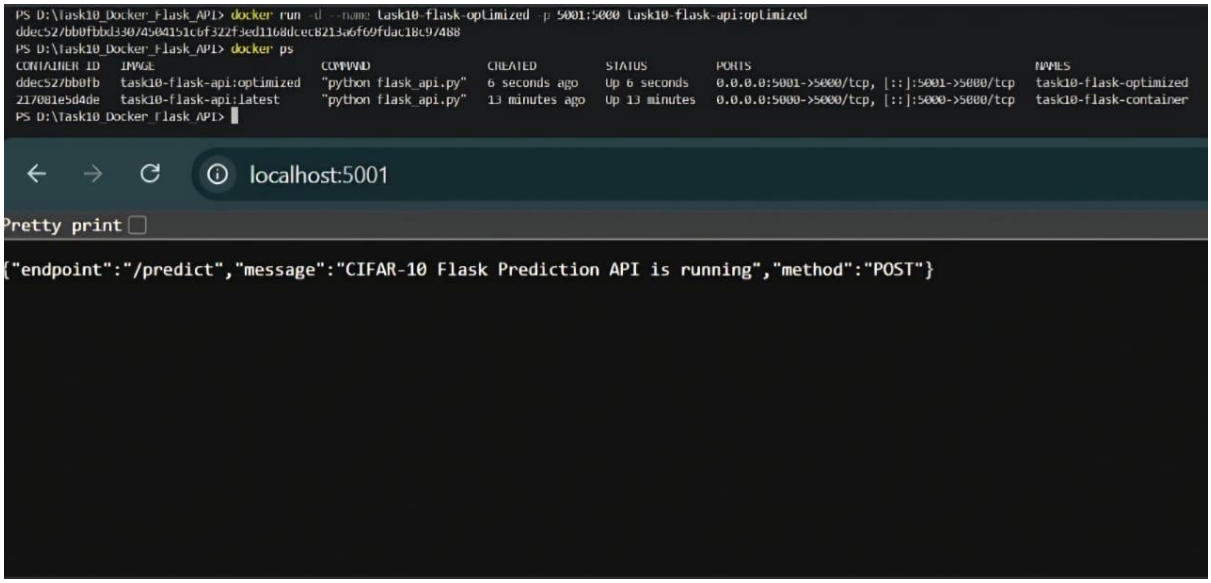


Figure 15: Optimized container listed by docker ps and health check at localhost:5001

The upper part of the screenshot shows the `docker run` command and the container listing. Two containers are now running side by side. The optimized container has the ID `ddec527bb0fb`, was created 6 seconds earlier and publishes `0.0.0.0:5001->5000/tcp`. The original container, `217081e5d4de`, has been up

for 13 minutes and still publishes port 5000. The lower part of the screenshot shows the browser at `localhost:5001` displaying the same JSON health-check response as before.

This confirms that the optimized image remained fully functional after rebuilding, and it also shows that several containers can be started from the same image and can run at the same time without interfering with each other. This is the basic mechanism by which a containerized service can be scaled.

15. Testing Results

This section summarizes every test that was carried out in the task. Each test has a clear pass condition and a result, and the last column points to the section that contains the evidence.

Table 25: Summary of test results

No.	Test	Expected outcome	Result	Section
1	Flask API local execution	Server starts on port 5000 and loads the model	Pass	5.8
2	Local API health check	JSON status message at `/`	Pass	5.9
3	Docker image build	Image `task10-flask-api:latest` is created	Pass	7
4	Docker container creation	Container is created from the image	Pass	8
5	Docker container execution	Status is Up and the command is `python flask_api.py`	Pass	8.2
6	Container port mapping	Host port 5000 forwards to container port 5000	Pass	8.2
7	Docker logs	Flask server visible on 0.0.0.0:5000	Pass	11
8	API health check in container	Same JSON message as the local run	Pass	9
9	CIFAR-10 image prediction	Frog image returns a valid class and confidence	Pass	10
10	Frog classification	Predicted class is Frog	Pass	10.3
11	Prediction confidence	Confidence of 99.01 %	Pass	10.3
12	Docker image history	Layers and sizes can be inspected	Pass	13
13	Optimized image build	Image `task10-flask-api:optimized` is created	Pass	14.1
14	Optimized container execution	Container runs on host port 5001	Pass	14.5
15	Optimized API health check	Health response at `localhost:5001`	Pass	14.5

15.1 Summary of Key Measurements

Table 26: Measurements recorded during the task

Measurement	Value
First build time	207.1 seconds
Time of the pip install step	151.8 seconds
Rebuild time using the cache	2.1 seconds
Build context size (first build)	3.83 MB
Image disk usage	3.11 GB
Image content size	710 MB
Dependency layer	2.25 GB
Model layer	3.84 MB
Application code layer	12.3 kB
Predicted class for the frog image	Frog
Confidence	99.01 %

All fifteen tests passed. The measurements show the two sides of the deployment: a large but stable dependency layer that dominates the size of the image and the build time, and a very small and fast application layer that changes rapidly during development.

16. Observations

The following observations were made while the task was carried out. They combine what was seen in the outputs with the reasoning that explains them.

16.1 Observations about Containerization

1. Docker packaged the Flask API, the Python dependencies and the trained deep learning model into one image, and no manual installation was needed to run it.
2. The containerized API was accessible through the mapped host port, and its behaviour was identical to the local run.
3. A relative model path (`model/cifar10_cnn.keras`) worked without change because the working directory of the container was set to `/app`.
4. Binding Flask to `0.0.0.0` was necessary for the port mapping to work.
5. Two containers were able to run from the same image at the same time by using different names and host ports.

16.2 Observations about Inference

1. TensorFlow performed inference using CPU execution, because no GPU was available to the container.
2. The CIFAR-10 frog image was classified as Frog with 99.01 % confidence.
3. The image of 611 × 407 pixels was reduced to 32 × 32 pixels and still produced a correct and confident prediction.
4. The messages about CUDA in the logs were informational for a CPU-only deployment and did not stop the application.

16.3 Observations about Size and Build Time

1. The TensorFlow dependency layer contributed the largest part of the final image size (2.25 GB).
2. The trained model occupied a comparatively small layer of 3.84 MB.
3. Docker layer caching significantly reduced the time required for the second build, from 207.1 seconds to 2.1 seconds.
4. The optimized image had the same reported size as the original image because the original Dockerfile already incorporated common optimization techniques.
5. The manifest and configuration digests of the two builds were identical, which confirms that the layers were the same.
6. The optimized image ran successfully as a separate Docker container.

16.4 Limitations

- The Flask development server is used, and Flask itself warns in the logs that it is not intended for production.
- The package versions in `requirements.txt` are not pinned, so a future rebuild may install different versions.

- The prediction test used one image. It shows that the pipeline works, but it does not measure the accuracy of the model.
- The container runs as the default user of the image, which is the root user, and no health-check instruction is defined in the Dockerfile.
- The image is large (3.11 GB) because of TensorFlow, and no attempt was made to replace it.

16.5 Lessons Learned

The most important lesson is that the design of the Dockerfile is at least as important as the code of the application. Small decisions, such as the order of two `COPY` instructions or a single flag on `pip install`, decide whether a build takes 2 seconds or more than 3 minutes. The second lesson is that measurement should come before optimization: the history of the image showed at once where the size comes from, and it made clear that no change in the application files could affect it. The third lesson is that logs should be read with understanding, since a line marked as an error may only record that an optional feature, such as a GPU, is not present.

17. Screenshots

The implementation was documented with 13 screenshots that follow the order of the work. The report also uses two additional images: the test image `frog.png` and the architecture diagram. Table 22 lists every screenshot with its file name, the figure in which it appears and the evidence that it provides. All the screenshots are embedded in the sections in which they are discussed.

Table 27: Index of screenshots

No.	Screenshot	Report figure	Section	Evidence provided
1	Flask API local running	Figure 2	5.8	API starts locally on port 5000
2	Local API health check	Figure 3	5.9	JSON status in the browser
3	Dockerfile	Figure 4	6	Content of the Dockerfile
4	Docker build success	Figure 5	7	Build in 207.1 s
5	Docker image created	Figure 6	7.2	Image ID and size
6	Docker container running	Figure 7	8.2	Output of `docker ps`
7	Docker container logs	Figure 9	11	Flask server start-up messages
8	Docker API health check	Figure 8	9	Health check through the container
9	CIFAR-10 prediction	Figure 11	10.3	Frog, 99.01 %
10	Image history	Figure 12	13	Layer sizes
11	Optimized image build	Figure 13	14.1	Build in 2.1 s with cached layers
12	Image size comparison	Figure 14	14.2	Both images 3.11 GB
13	Optimized container API check	Figure 15	14.5	Container on port 5001

In addition to the screenshots, Figure 1 shows the architecture of the system and Figure 10 shows the test image that was submitted to the prediction endpoint.

17.1 Value of Each Type of Evidence

- **Terminal screenshots** (build, `docker ps`, `docker images`, logs, history) prove that the commands were executed and record the exact identifiers, timings and sizes.

- **Browser screenshots** (health checks) prove that the API is reachable over HTTP from the host.
- **The prediction screenshot** proves that the whole chain, from the uploaded file to the JSON answer, works inside the container.
- **The Dockerfile and editor screenshots** show the configuration that produced the results, so that the results can be reproduced.

18. Conclusion

This task successfully demonstrated the containerization of a Flask-based deep learning API using Docker. The trained CIFAR-10 CNN model was packaged together with the Flask application and the required Python dependencies. A Docker image was created and used to launch the API inside a container.

The API was verified through a health-check endpoint and through an actual CIFAR-10 image prediction. The test image was correctly classified as Frog with a confidence of 99.01 %. The logs confirmed that the Flask server started on 0.0.0.0:5000 and that TensorFlow used CPU execution because no GPU was available.

The image history was inspected to understand the composition of the image. It showed that the dependency layer of 2.25 GB is the dominant contributor to the size, while the model occupies only 3.84 MB. The optimization practices applied in the Dockerfile, namely the slim Python base image, removal of the pip cache, layer caching through the order of the instructions and the use of `.dockerignore`, were explained and evaluated. An optimized image was built in 2.1 seconds compared with 207.1 seconds for the first build, and it was run successfully as a separate container on host port 5001.

18.1 Mapping to the Task Criteria

Table 28: Final status against the evaluation criteria

Criterion	Outcome	Status
Correct containerization	Dockerfile with ordered, documented instructions; image built successfully	Achieved
Successful deployment	Container running, port mapped, health check and prediction verified	Achieved
Optimization practices	Slim base, no pip cache, layer caching, `.dockerignore`, image history analysis and optimized rebuild	Achieved

18.2 Final Status

Task 10 Status: Completed Successfully. The Flask deep learning API was containerized, built as a Docker image, executed inside a Docker container, tested through the API, verified with a CIFAR-10 prediction, inspected with the Docker image history, rebuilt with optimized Docker practices and successfully executed using the optimized image.

Appendix A: Source Code Listings

This appendix collects the complete source files of the project in one place for reference, so that the reader does not need to search through the sections for a specific file.

A.1 flask_api.py

```
from flask import Flask, request, jsonify
import tensorflow as tf
import numpy as np
from PIL import Image
import io

# ==========================================
# Flask Application
# ==========================================

app = Flask(__name__)


# ==========================================
# CIFAR-10 Class Names
# ==========================================

class_names = [
    "Airplane",
    "Automobile",
    "Bird",
    "Cat",
    "Deer",
    "Dog",
    "Frog",
    "Horse",
    "Ship",
    "Truck"
]


# ==========================================
# Load Trained CNN Model
# ==========================================

MODEL_PATH = "model/cifar10_cnn.keras"

model = tf.keras.models.load_model(MODEL_PATH)


# ==========================================
# Home / Health Check Endpoint
# ==========================================

@app.route("/", methods=["GET"])
def home():
    return jsonify({
        "message": "CIFAR-10 Flask Prediction API is running",
        "endpoint": "/predict",
        "method": "POST"
    })
```

```
# ========================================
# Prediction Endpoint
# ========================================

@app.route("/predict", methods=["POST"])
def predict():

    if "image" not in request.files:
        return jsonify({
            "error": "No image provided. Please upload an image."
        }), 400

    try:
        image_file = request.files["image"]
        image = Image.open(image_file).convert("RGB")
        image = image.resize((32, 32))
        image_array = np.array(image)
        image_array = image_array.astype("float32") / 255.0
        image_array = np.expand_dims(image_array, axis=0)

        predictions = model.predict(
            image_array,
            verbose=0
        )

        predicted_index = np.argmax(predictions[0])
        predicted_class = class_names[predicted_index]
        confidence = float(predictions[0][predicted_index]) * 100

        return jsonify({
            "predicted_class": predicted_class,
            "confidence": round(confidence, 2)
        })

    except Exception as e:
        return jsonify({
            "error": str(e)
        }), 500


# ========================================
# Run Flask Application
# ========================================

if __name__ == "__main__":
    app.run(
        host="0.0.0.0",
        port=5000,
        debug=False
    )
```

A.2 Dockerfile

```
# Use a lightweight Python base image
FROM python:3.11-slim

# Set the working directory inside the container
WORKDIR /app

# Copy requirements first for better Docker layer caching
COPY requirements.txt .
```

```
# Install Python dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Copy Flask application
COPY flask_api.py .

# Copy trained deep learning model
COPY model ./model

# Expose Flask API port
EXPOSE 5000

# Start Flask API
CMD ["python", "flask_api.py"]
```

A.3 requirements.txt

```
Flask
tensorflow
numpy
Pillow
```

A.4 .dockerignore

```
__pycache__
*.pyc
*.pyo
*.pyd
.git
.gitignore
.vscode
screenshots
screenshots9
README.md
```

Appendix B: Docker Command Reference

This appendix lists every Docker and shell command that was used in the task, in the order in which they appear in the report, together with a short description.

Table 29: Complete command reference

Command	Description
`docker build -t task10-flask-api:latest .`	Builds the image from the Dockerfile in the current folder
`docker images`	Lists local images with their ID, disk usage and content size
`docker run -d --name task10-flask-container -p 5000:5000 task10-flask-api:latest`	Starts a detached container and publishes port 5000
`docker ps`	Lists running containers
`docker logs task10-flask-container`	Prints the collected output of the container
`curl.exe -X POST -F "image=@frog.png" http://localhost:5000/predict`	Uploads an image to the prediction endpoint
`docker image history task10-flask-api:latest`	Shows the layers of the image and their sizes
`docker build -t task10-flask-api:optimized .`	Rebuilds the image under a new tag, reusing cached layers
`docker run -d --name task10-flask-optimized -p 5001:5000 task10-flask-api:optimized`	Starts a second container from the optimized image on host port 5001

B.1 Additional Useful Commands

The commands below were not required to complete the task but are useful when working with the same project, for example while writing this report or while cleaning up afterwards.

Table 30: Useful commands beyond the task

Command	Description
`docker stop task10-flask-container task10-flask-optimized`	Stops both running containers

Command	Description
`docker rm task10-flask-container task10-flask-optimized`	Removes the stopped containers
`docker rmi task10-flask-api:optimized`	Removes the optimized image tag while keeping the shared layers used by `latest`
`docker system df`	Shows a summary of the space used by images, containers and the build cache
`docker exec -it task10-flask-container /bin/bash`	Opens an interactive shell inside a running container for debugging
`docker inspect task10-flask-container`	Prints detailed configuration and state of a container as JSON

Appendix C: Troubleshooting Notes

Although this task completed without a failed test, the following notes record issues that can occur with the same kind of project and how the evidence in this report would help to diagnose them.

Table 31: Common issues and how to diagnose them

Symptom	Likely cause	How to check
`curl` in PowerShell gives an unexpected result or a parameter error	PowerShell defines `curl` as an alias for `Invoke-WebRequest`, whose syntax differs from real curl	Use `curl.exe` explicitly, as in Section 10.2
Browser cannot reach `localhost:5000`	The container is not running, or the port was not published, or Flask is bound to `127.0.0.1`	Check `docker ps` for the port mapping, and confirm `app.run(host="0.0.0.0", ...)` in the code
Container starts and then exits immediately	The model file failed to load, often because of a wrong path or a missing file	Run `docker logs` and look for a traceback near `load_model`
`docker run` fails with a name conflict	A container with the same `--name` already exists	Remove the old container with `docker rm`, or choose a new name, as was done for the optimized container
Build re-downloads TensorFlow even though the code alone changed	A file that is copied before `pip install` changed, or `.dockerignore` is missing	Confirm the copy order in the Dockerfile and check that `requirements.txt` itself is unchanged
Prediction endpoint returns an error with status 400	The multipart form field is not named `image`	Check the `-F` option of curl or the field name used by the client
Prediction endpoint returns an error with status 500	The uploaded file could not be opened as an image	Read the `error` field of the JSON response, which contains the Python exception message

C.1 Reading TensorFlow Start-up Warnings

New users of TensorFlow are often concerned by the number of warnings that are printed when the library starts. Section 11.2 explains that in a CPU-only container, the CUDA-related messages describe an expected and harmless condition. As a general rule, a start-up warning should be treated as a problem only if the specific feature it refers to is required. In this task no GPU was required, so the CUDA warnings could be safely ignored.

Appendix D: Suggestions for Future Work

The task was scoped as a demonstration of containerization, and it fulfils that scope completely. If the same project were to move toward a production deployment, the following improvements would be the natural next steps, in an order that reflects their impact relative to their cost.

D.1 Serving and Reliability

1. **Use a production WSGI server**, such as Gunicorn, behind Flask, so that the server can handle concurrent requests safely and does not print the development-server warning seen in the logs.
2. **Add a Docker health-check instruction** (`HEALTHCHECK`) so that Docker itself can detect a container that is running but not responding, and restart it automatically under an orchestrator.
3. **Run the container as a non-root user** created with a `USER` instruction, which reduces the impact of a potential vulnerability inside the container.
4. **Pin the versions** of Flask, TensorFlow, NumPy and Pillow in `requirements.txt`, so that a rebuild next year installs the same versions that were tested in this report.

D.2 Size and Performance

1. **Evaluate `tensorflow-cpu`** in place of `tensorflow`, since the container never uses a GPU; if the package is smaller, the largest layer of the image would shrink directly.
2. **Evaluate a multi-stage build** if any compiled dependency is added later, so that build-time tools do not remain in the final image.
3. **Batch or cache repeated predictions** if the API is expected to serve the same or similar images often.
4. **Consider TensorFlow Lite** for the trained model if the deployment target has limited storage or memory, at the cost of a conversion step and re-validation of accuracy.

D.3 Operations

1. **Add structured logging** with request identifiers, so that a specific prediction can be traced through the logs.
2. **Add automated tests** that build the image, start a container and send a request as part of a continuous-integration pipeline.
3. **Publish the image to a registry** such as Docker Hub so that it can be pulled directly on a deployment server instead of being rebuilt there.
4. **Add resource limits** (`--memory`, `--cpus`) to the `docker run` command so that the container cannot exhaust the resources of the host.

Appendix E: Glossary

Table 32: Glossary of terms used in this report

Term	Definition
API	Application Programming Interface; a defined way for one program to request a service from another
CNN	Convolutional Neural Network; a type of neural network well suited to image data
CIFAR-10	A dataset of 60,000 small colour images in ten classes, commonly used to test image classifiers
Container	A running, isolated instance of a Docker image
Dockerfile	A text file of instructions that describes how to build a Docker image
Image	A read-only template, made of layers, from which containers are created
Layer	The file-system change produced by one instruction of a Dockerfile
Layer caching	Reuse of a previously built layer when its instruction and inputs have not changed
Port mapping	A rule, written `host:container`, that forwards traffic from a host port into a container
REST	Representational State Transfer; an architectural style for web APIs based on standard HTTP methods
Softmax	An activation function that turns a vector of numbers into probabilities that sum to one
Tag	A label attached to an image, such as `latest` or `optimized`, usually alongside a version or variant name
WSGI	Web Server Gateway Interface; the standard interface between Python web applications and production web servers